



UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO

FACULTAD DE INGENIERÍA

INGENIERÍA EN COMPUTACIÓN

Laboratorio de Organización de Arquitectura de Computadoras

Reporte - Construcción de máquinas de Estados usando memorias: Direccionamiento por Trayectoria

NOMBRE DE LOS INTEGRANTES:

Omar Alfonso Zendejas Labias Grupo de Teoría: 04

Emiliano García Olvera Grupo de Teoría: 02

NOMBRE DEL PROFESOR: ING. KAREN PÉREZ

GRUPO: 06 EQUIPO: 9

SEMESTRE: 2027-1.

FECHA DE ENTREGA: 08 de octubre de 2026

Objetivo

Construir una máquina de estados finita a partir de una carta ASM empleando una memoria como elemento central del diseño, bajo el método de direccionamiento por trayectoria. A diferencia del diseño clásico con *flip-flops* y ecuaciones booleanas obtenidas por mapas de Karnaugh, aquí la lógica de estado siguiente y de salidas no se sintetiza como compuertas, sino que se almacena explícitamente en localidades de memoria.

Para ello se diseñó una carta ASM de cinco estados, tres entradas y cuatro salidas (dos con lógica estándar y dos con lógica negada), se determinó el tamaño de memoria requerido, se obtuvo su contenido completo y se implementó el sistema en VHDL sobre la tarjeta de desarrollo DE10-Lite mediante Quartus Prime, verificando su funcionamiento en hardware.

Introducción

Una carta ASM (*Algorithmic State Machine*) es una representación gráfica del comportamiento de un circuito secuencial que se asemeja a un diagrama de flujo: los estados se dibujan como cajas rectangulares, las condiciones de transición como cajas de decisión y las salidas que dependen simultáneamente del estado y de las entradas como cajas ovaladas, denominadas salidas condicionales. Esta distinción corresponde a los dos modelos clásicos de máquina secuencial: en el modelo de Moore las salidas son función únicamente del estado presente, mientras que en el modelo de Mealy dependen también de las entradas, por lo que pueden cambiar sin que medie un flanco de *clock* [2].

El método de implementación empleado en esta práctica sustituye la red combinacional de estado siguiente por una memoria. En el **direccionamiento por trayectoria**, cada localidad de memoria guarda dos campos: la *liga*, que codifica la dirección del estado siguiente, y la *parte de las salidas*, que contiene el valor de las señales de salida [1]. La dirección con la que se consulta la memoria se forma concatenando la liga del estado presente con las variables de entrada; la palabra leída se divide nuevamente en liga y salidas, y la liga se almacena en un registro cuya salida realimenta las líneas de dirección. La señal de reloj conectada a ese registro determina la velocidad a la que avanza la máquina de estados.

Puesto que la dirección se compone de los bits de estado más los bits de entrada, el tamaño de la memoria queda determinado por

$$\text{Memoria} = 2^n \times m \quad (1)$$

donde n es el número total de bits de dirección ($n = \text{bits de estado} + \text{bits de entrada}$) y m el número de bits almacenados por localidad ($m = \text{bits de liga} + \text{bits de salidas}$). Debe notarse que la memoria contempla *todas* las combinaciones posibles de las variables de entrada para cada estado, incluso aquellas que ese estado no evalúa; en tales casos las localidades correspondientes repiten la misma liga, ya que la transición es independiente de esas variables [1].

La ventaja de este esquema es que modificar el comportamiento de la máquina no requiere rediseñar lógica combinacional, sino únicamente reescribir el contenido de la memoria. A cambio, el costo en bits crece exponencialmente con el número de entradas, pues cada entrada adicional duplica la cantidad de localidades necesarias.

Desarrollo

Planteamiento: carta ASM

Se diseñó una carta ASM que cumple con las características solicitadas en el manual: cinco estados, tres entradas (Q_1 , Q_2 y Q_3) y cuatro salidas, de las cuales S_1 y S_2 operan con lógica estándar (activas en 1) y S_3 y S_4 con lógica negada (activas en 0, lo que se indica con una barra sobre el nombre de la señal). Adicionalmente se incorporaron salidas condicionales en ambos caminos de una decisión, tal como lo exige el punto 1 del manual. El desarrollo original realizado a mano se muestra en la figura 1, y su versión redibujada en limpio, que será la referencia para el resto del reporte, en la figura 2.



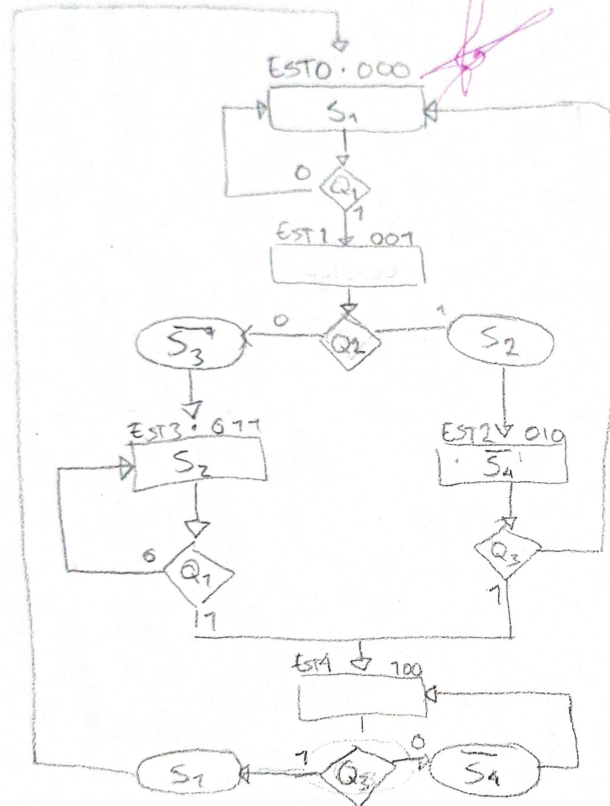
UNIVERSIDAD NACIONAL AUTONOMA DE MEXICO

EXAMEN: _____

PROFESOR: _____

MATERIA: _____

NOMBRE DEL ALUMNO: _____



Tamaño de la memoria en bits: $(\# \text{Estados} + 2^{\# \text{Entradas}}) (\log_2 \# \text{Estados}) + \# \text{Salidas}$

$\# \text{Estados} = 5$
 $\# \text{Entradas} = 3 \therefore 2^3 = 8$
 $\log_2(5) = 3 \text{ bits de log}$
 $\# \text{Salidas} = 4$

Memoria: $(5 + 8) + (5 \times 4) = 40 + 20 = 60 \text{ bits}$
 es decir, 40 localidades de 7 bits
 cada una (5 de log y 4 de salida)

Figura 1: Desarrollo manuscrito de la carta ASM y del cálculo del tamaño de memoria.

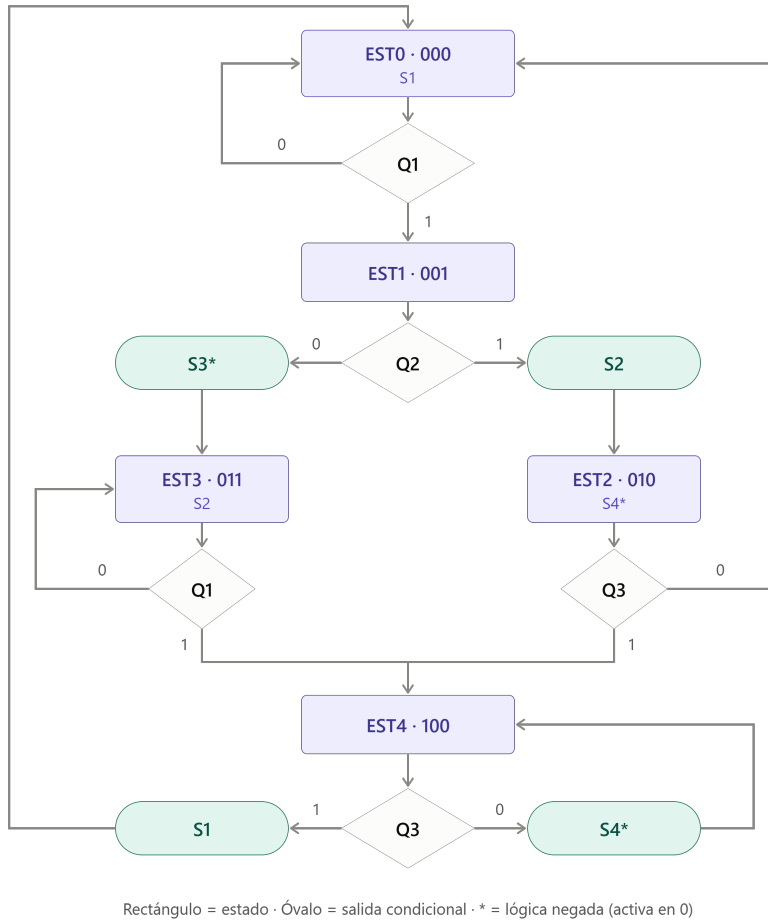


Figura 2: Carta ASM propuesta: cinco estados, tres entradas y cuatro salidas, con salidas condicionales en ambos caminos de las decisiones de EST1 y EST4.

Antes de construir la tabla de transiciones fue necesario asignar a cada estado una representación binaria. Dado que la carta tiene cinco estados, se requieren $\lceil \log_2 5 \rceil = 3$ bits, con lo que la asignación queda como se indica en la tabla 1. Los tres códigos restantes (101, 110 y 111) no corresponden a ningún estado de la carta.

Tabla 1: Asignación binaria de los estados y salida incondicional asociada a cada uno.

Estado	Código	Salida incondicional	Variable evaluada
EST0	000	S_1	Q_1
EST1	001	—	Q_2
EST2	010	$\overline{S_4}$	Q_3
EST3	011	S_2	Q_1
EST4	100	—	Q_3

El comportamiento de la máquina es el siguiente. En EST0 se activa S_1 de forma incondicional y la máquina permanece en ese estado mientras $Q_1 = 0$; cuando $Q_1 = 1$ avanza a EST1. EST1 no activa ninguna salida incondicional y evalúa Q_2 : por la rama falsa se activa la salida condicional $\overline{S_3}$ y se transita a EST3, mientras que por la rama verdadera se activa la salida condicional S_2 y se transita a EST2. En EST2 se activa $\overline{S_4}$ y, según Q_3 , la máquina regresa a EST0 ($Q_3 = 0$) o avanza a EST4 ($Q_3 = 1$). EST3 activa S_2 y se autolaza mientras $Q_1 = 0$, avanzando a EST4 cuando $Q_1 = 1$. Finalmente, EST4 evalúa Q_3 con salidas condicionales en ambos caminos: con $Q_3 = 0$ activa $\overline{S_4}$ y se autolaza, y con $Q_3 = 1$ activa S_1 y retorna a EST0.

Respecto a la codificación de las salidas negadas, se adoptó el criterio de almacenar en memoria el *nivel físico* de la señal: el bit correspondiente a $\overline{S_3}$ o $\overline{S_4}$ vale 0 cuando la salida está activa y 1 cuando se encuentra en reposo, de manera inversa a S_1 y S_2 . Con ello la palabra leída de la memoria se conecta directamente a los pines de salida, sin inversores intermedios.

Tamaño de la memoria necesaria

De acuerdo con la ecuación 1, la dirección se forma con los 3 bits del estado presente y los 3 bits de entrada, por lo que $n = 3 + 3 = 6$. Cada localidad almacena los 3 bits de la liga y los 4 bits de salidas, de modo que $m = 3 + 4 = 7$. Por lo tanto

$$\text{Memoria} = 2^6 \times 7 = 64 \times 7 = 448 \text{ bits},$$

es decir, 64 localidades de 7 bits cada una. De esas 64 localidades, únicamente $5 \times 2^3 = 40$ corresponden a estados que existen en la carta ASM; las 24 restantes pertenecen a los códigos 101, 110 y 111, que no se utilizan. Dichas localidades no pueden omitirse del dispositivo físico, puesto que un bus de direcciones de 6 bits las hace accesibles, por lo que se les asignó un contenido seguro: liga hacia EST0 y salidas en reposo, de forma que si la máquina llegara a uno de esos códigos por una condición anómala, retornaría al estado inicial en el siguiente flanco de *clock*.

Conciliar con el cálculo manuscrito de la figura 1, que reporta 280 bits (40 localidades $\times$ 7). Ese número corresponde a las localidades efectivamente utilizadas; la fórmula $2^n \times m$ del manual, con $n = 6$, da 448 bits. Definir con el equipo cuál de los dos se reporta como respuesta al punto 2.

Contenido de la memoria

El contenido completo de la memoria se obtuvo recorriendo la carta ASM estado por estado y, para cada uno, enumerando las ocho combinaciones posibles de $Q_1Q_2Q_3$. La dirección se forma como

$$\text{dir} = \underbrace{\text{Edo. presente}}_{\text{dir}(5:3)} \parallel \underbrace{Q_1 Q_2 Q_3}_{\text{dir}(2:0)} \quad \text{y} \quad \text{dato} = \underbrace{\text{Liga}}_{\text{dato}(6:4)} \parallel \underbrace{S_1 S_2 S_3 S_4}_{\text{dato}(3:0)}$$

En los estados que evalúan una sola variable, las localidades correspondientes a las combinaciones que difieren únicamente en las variables no evaluadas comparten la misma liga y las mismas salidas. Por ejemplo, EST0 solo depende de Q_1 , de modo que las direcciones 0 a 3 ($Q_1 = 0$) tienen liga 000 y las direcciones 4 a 7 ($Q_1 = 1$) tienen liga 001, sin importar el valor de Q_2 y Q_3 . La tabla 2 presenta las 64 localidades resultantes.

Implementación en VHDL

El sistema se describió en VHDL siguiendo la estructura de la figura 1 del manual, repartido en cinco módulos más una entidad de nivel superior que los interconecta. El flujo de datos es el siguiente: el estado presente y las entradas se unen en el concatenador para formar la dirección; la memoria entrega la palabra de 7 bits; el divisor de datos la separa en liga y salidas; y la liga regresa al registro de estado, que la captura en el siguiente flanco del reloj dividido.

El módulo concatenador_datos, mostrado en la figura 3, une el estado presente en la parte alta de la dirección y las entradas en la parte baja.

```
1 library IEEE;
2 use IEEE.STD_LOGIC_1164.ALL;
3 use IEEE.STD_LOGIC_ARITH.ALL;
4 use IEEE.STD_LOGIC_UNSIGNED.ALL;
5
6 entity concatenador_datos is
7     Port( entradaA : in STD_LOGIC_VECTOR(2 downto 0); --Entradas
8           entradaB : in STD_LOGIC_VECTOR(2 downto 0); --Edo Presente
9           salida : out STD_LOGIC_VECTOR(5 downto 0)
10        );
11 end concatenador_datos;
12
13 architecture Behavioral of concatenador_datos is
14 begin
```

Tabla 2: Contenido de memoria obtenido por direccionamiento por trayectoria. Las direcciones 40 a 63 corresponden a los códigos de estado no utilizados.

Dir.	Edo.	Q ₁ Q ₂ Q ₃	Liga	S ₁ S ₂ S ₃ S ₄
0	000	000	000	1011
1	000	001	000	1011
2	000	010	000	1011
3	000	011	000	1011
4	000	100	001	1011
5	000	101	001	1011
6	000	110	001	1011
7	000	111	001	1011
8	001	000	011	0001
9	001	001	011	0001
10	001	010	010	0111
11	001	011	010	0111
12	001	100	011	0001
13	001	101	011	0001
14	001	110	010	0111
15	001	111	010	0111
16	010	000	000	0010
17	010	001	100	0010
18	010	010	000	0010
19	010	011	100	0010
20	010	100	000	0010
21	010	101	100	0010
22	010	110	000	0010
23	010	111	100	0010
24	011	000	011	0111
25	011	001	011	0111
26	011	010	011	0111
27	011	011	011	0111
28	011	100	100	0111
29	011	101	100	0111
30	011	110	100	0111
31	011	111	100	0111

Dir.	Edo.	Q ₁ Q ₂ Q ₃	Liga	S ₁ S ₂ S ₃ S ₄
32	100	000	100	0010
33	100	001	000	1011
34	100	010	100	0010
35	100	011	000	1011
36	100	100	100	0010
37	100	101	000	1011
38	100	110	100	0010
39	100	111	000	1011
40	101	000	000	0011
41	101	001	000	0011
42	101	010	000	0011
43	101	011	000	0011
44	101	100	000	0011
45	101	101	000	0011
46	101	110	000	0011
47	101	111	000	0011
48	110	000	000	0011
49	110	001	000	0011
50	110	010	000	0011
51	110	011	000	0011
52	110	100	000	0011
53	110	101	000	0011
54	110	110	000	0011
55	110	111	000	0011
56	111	000	000	0011
57	111	001	000	0011
58	111	010	000	0011
59	111	011	000	0011
60	111	100	000	0011
61	111	101	000	0011
62	111	110	000	0011
63	111	111	000	0011


```

15 process (entradaA, entradaB)
16 begin
17     --Asignar la concatenacion del Edo Presente y Entradas a la Salida.
18     -- salida(5..3) = Edo Presente, salida(2..0) = Q1 Q2 Q3
19     salida <= entradaB & entradaA;
20 end process;
21 end Behavioral;

```

Figura 3: Código VHDL del concatenador que forma la dirección de memoria.

El módulo memory implementa la ROM de 64 palabras de 7 bits. Cada asignación corresponde a un renglón de la tabla 2, agrupada por estado y comentada con la combinación de entradas que le da origen. El código se muestra en la figura 4.

```

1 library IEEE;
2 use IEEE.STD_LOGIC_1164.ALL;
3 use IEEE.STD_LOGIC_ARITH.ALL;
4 use IEEE.STD_LOGIC_UNSIGNED.ALL;
5
6 entity memory is
7     Port( dir : in STD_LOGIC_VECTOR(5 downto 0); --Entradas + edo presente
8           data : out STD_LOGIC_VECTOR(6 downto 0) --Liga + Salidas
9         );
10 end memory;
11
12 architecture Behavioral of memory is
13     --Arreglo ROM de 64 palabras de 7 bits (Memoria = 2^6 x 7 = 448 bits)
14     type rom_type is array (0 to 63) of std_logic_vector(6 downto 0);
15     signal internal_mem : rom_type;
16
17 begin
18     -- dir = Edo.Presente(2..0) & Q1 & Q2 & Q3
19     -- data = Liga(L2 L1 L0) & Salidas(S1 S2 S3 S4)
20     -- S1 y S2: logica estandar (1 = activa)
21     -- S3 y S4: logica negada (0 = activa)
22
23     -- EST0 (000) -- direcciones 0 a 7
24     internal_mem( 0) <= "000" & "1011"; -- Q1Q2Q3=000 -> liga 000 / S1
25     internal_mem( 1) <= "000" & "1011"; -- Q1Q2Q3=001 -> liga 000 / S1
26     internal_mem( 2) <= "000" & "1011"; -- Q1Q2Q3=010 -> liga 000 / S1
27     internal_mem( 3) <= "000" & "1011"; -- Q1Q2Q3=011 -> liga 000 / S1
28     internal_mem( 4) <= "001" & "1011"; -- Q1Q2Q3=100 -> liga 001 / S1
29     internal_mem( 5) <= "001" & "1011"; -- Q1Q2Q3=101 -> liga 001 / S1
30     internal_mem( 6) <= "001" & "1011"; -- Q1Q2Q3=110 -> liga 001 / S1
31     internal_mem( 7) <= "001" & "1011"; -- Q1Q2Q3=111 -> liga 001 / S1

```

```

32
33  -- EST1 (001) -- direcciones 8 a 15
34  internal_mem( 8) <= "011" & "0001"; -- Q1Q2Q3=000 -> liga 011 / S3* cond.
35  internal_mem( 9) <= "011" & "0001"; -- Q1Q2Q3=001 -> liga 011 / S3* cond.
36  internal_mem(10) <= "010" & "0111"; -- Q1Q2Q3=010 -> liga 010 / S2 cond.
37  internal_mem(11) <= "010" & "0111"; -- Q1Q2Q3=011 -> liga 010 / S2 cond.
38  internal_mem(12) <= "011" & "0001"; -- Q1Q2Q3=100 -> liga 011 / S3* cond.
39  internal_mem(13) <= "011" & "0001"; -- Q1Q2Q3=101 -> liga 011 / S3* cond.
40  internal_mem(14) <= "010" & "0111"; -- Q1Q2Q3=110 -> liga 010 / S2 cond.
41  internal_mem(15) <= "010" & "0111"; -- Q1Q2Q3=111 -> liga 010 / S2 cond.
42
43  -- EST2 (010) -- direcciones 16 a 23
44  internal_mem(16) <= "000" & "0010"; -- Q1Q2Q3=000 -> liga 000 / S4*
45  internal_mem(17) <= "100" & "0010"; -- Q1Q2Q3=001 -> liga 100 / S4*
46  internal_mem(18) <= "000" & "0010"; -- Q1Q2Q3=010 -> liga 000 / S4*
47  internal_mem(19) <= "100" & "0010"; -- Q1Q2Q3=011 -> liga 100 / S4*
48  internal_mem(20) <= "000" & "0010"; -- Q1Q2Q3=100 -> liga 000 / S4*
49  internal_mem(21) <= "100" & "0010"; -- Q1Q2Q3=101 -> liga 100 / S4*
50  internal_mem(22) <= "000" & "0010"; -- Q1Q2Q3=110 -> liga 000 / S4*
51  internal_mem(23) <= "100" & "0010"; -- Q1Q2Q3=111 -> liga 100 / S4*
52
53  -- EST3 (011) -- direcciones 24 a 31
54  internal_mem(24) <= "011" & "0111"; -- Q1Q2Q3=000 -> liga 011 / S2
55  internal_mem(25) <= "011" & "0111"; -- Q1Q2Q3=001 -> liga 011 / S2
56  internal_mem(26) <= "011" & "0111"; -- Q1Q2Q3=010 -> liga 011 / S2
57  internal_mem(27) <= "011" & "0111"; -- Q1Q2Q3=011 -> liga 011 / S2
58  internal_mem(28) <= "100" & "0111"; -- Q1Q2Q3=100 -> liga 100 / S2
59  internal_mem(29) <= "100" & "0111"; -- Q1Q2Q3=101 -> liga 100 / S2
60  internal_mem(30) <= "100" & "0111"; -- Q1Q2Q3=110 -> liga 100 / S2
61  internal_mem(31) <= "100" & "0111"; -- Q1Q2Q3=111 -> liga 100 / S2
62
63  -- EST4 (100) -- direcciones 32 a 39
64  internal_mem(32) <= "100" & "0010"; -- Q1Q2Q3=000 -> liga 100 / S4* cond.
65  internal_mem(33) <= "000" & "1011"; -- Q1Q2Q3=001 -> liga 000 / S1 cond.
66  internal_mem(34) <= "100" & "0010"; -- Q1Q2Q3=010 -> liga 100 / S4* cond.
67  internal_mem(35) <= "000" & "1011"; -- Q1Q2Q3=011 -> liga 000 / S1 cond.
68  internal_mem(36) <= "100" & "0010"; -- Q1Q2Q3=100 -> liga 100 / S4* cond.
69  internal_mem(37) <= "000" & "1011"; -- Q1Q2Q3=101 -> liga 000 / S1 cond.
70  internal_mem(38) <= "100" & "0010"; -- Q1Q2Q3=110 -> liga 100 / S4* cond.
71  internal_mem(39) <= "000" & "1011"; -- Q1Q2Q3=111 -> liga 000 / S1 cond.
72
73  -- Estados no utilizados (101, 110, 111) -- regresan a EST0, salidas en
    reposo
74  internal_mem(40) <= "000" & "0011";

```

```

75  internal_mem(41) <= "000" & "0011";
76  internal_mem(42) <= "000" & "0011";
77  internal_mem(43) <= "000" & "0011";
78  internal_mem(44) <= "000" & "0011";
79  internal_mem(45) <= "000" & "0011";
80  internal_mem(46) <= "000" & "0011";
81  internal_mem(47) <= "000" & "0011";
82  internal_mem(48) <= "000" & "0011";
83  internal_mem(49) <= "000" & "0011";
84  internal_mem(50) <= "000" & "0011";
85  internal_mem(51) <= "000" & "0011";
86  internal_mem(52) <= "000" & "0011";
87  internal_mem(53) <= "000" & "0011";
88  internal_mem(54) <= "000" & "0011";
89  internal_mem(55) <= "000" & "0011";
90  internal_mem(56) <= "000" & "0011";
91  internal_mem(57) <= "000" & "0011";
92  internal_mem(58) <= "000" & "0011";
93  internal_mem(59) <= "000" & "0011";
94  internal_mem(60) <= "000" & "0011";
95  internal_mem(61) <= "000" & "0011";
96  internal_mem(62) <= "000" & "0011";
97  internal_mem(63) <= "000" & "0011";
98
99  process(dir, internal_mem)
100  begin
101      data <= internal_mem(conv_integer(unsigned(dir))); --conversion de
102                                     palabra a entero
103  end process;
104 end Behavioral;

```

Figura 4: Código VHDL de la memoria con las 64 localidades del direccionamiento por trayectoria.

El módulo `divisor_datos` de la figura 5 realiza la operación inversa al concatenador: separa los tres bits más significativos de la palabra leída como `liga` y los cuatro menos significativos como `salidas`.

```

1  library IEEE;
2  use IEEE.STD_LOGIC_1164.ALL;
3  use IEEE.STD_LOGIC_ARITH.ALL;
4  use IEEE.STD_LOGIC_UNSIGNED.ALL;
5
6  entity divisor_datos is
7      Port( entrada : in STD_LOGIC_VECTOR(6 downto 0); --Entradas
8            liga : out STD_LOGIC_VECTOR(2 downto 0); --Edo Presente

```

```

9      salidas : out STD_LOGIC_VECTOR(3 downto 0)
10    );
11 end divisor_datos;
12
13 architecture Behavioral of divisor_datos is
14 begin
15   process (entrada)
16   begin
17     liga <= entrada(6 downto 4);    --Los 3 bits mas significativos de la
18                                     entrada
19     salidas <= entrada(3 downto 0); --Los 4 bits menos significativos de la
20                                     entrada
21   end process;
22 end Behavioral;

```

Figura 5: Código VHDL del divisor que separa la liga de las salidas.

El registro de estado, mostrado en la figura 6, es el único elemento secuencial del lazo principal. Cuenta con un *reset* asíncrono activo en bajo que fuerza el estado 000 (EST0), y en cada flanco de subida captura la liga entregada por la memoria.

```

1 library IEEE;
2 use IEEE.STD_LOGIC_1164.ALL;
3
4 entity register2 is
5   Port( CLK : in STD_LOGIC;
6         RESET : in STD_LOGIC;
7         DATA_IN : in STD_LOGIC_VECTOR(2 downto 0); --Liga
8         DATA_OUT : out STD_LOGIC_VECTOR(2 downto 0) --Edo Presente
9   );
10 end register2;
11
12 architecture Behavioral of register2 is
13   signal internal_value : std_logic_vector(2 downto 0) := B"000";
14 begin
15   process (CLK, RESET)
16   begin
17     if RESET = '0' then
18       internal_value <= B"000";    --EST0
19     elsif rising_edge (CLK) then
20       internal_value <= DATA_IN;   --Liga del estado presente
21     end if;
22   end process;
23
24   process(internal_value)

```

```

25  begin
26      DATA_OUT <= internal_value;
27  end process;
28  end Behavioral;

```

Figura 6: Código VHDL del registro que almacena el estado presente.

Dado que el oscilador de la tarjeta opera a 50 MHz, las transiciones resultarían imperceptibles para el observador. Por ello se incluyó el divisor de frecuencia de la figura 7, que genera el reloj con el que avanza la máquina. El contador se reinicia al alcanzar el valor `X"2FAF080"` (50 000 000) y la salida se toma del bit 25, lo que produce un periodo de $50\,000\,001/50 \times 10^6 \approx 1.0$ s; es decir, **una transición de estado por segundo**.

```

1  library IEEE;
2  use IEEE.STD_LOGIC_1164.ALL;
3  use IEEE.STD_LOGIC_ARITH.ALL;
4  use IEEE.STD_LOGIC_UNSIGNED.ALL;
5
6  entity div_frec is
7      Port (reloj : in std_logic;
8            div_clk : out std_logic);
9  end div_frec;
10
11  architecture Behavioral of div_frec is
12  begin
13      process (reloj)
14          variable cuenta : std_logic_vector (27 downto 0) := X"00000000";
15          begin
16              if rising_edge (reloj) then
17
18                  --if cuenta 0 "40000000" then
19                  if cuenta = X"2FAF080" then
20                      cuenta := X"00000000";
21                  else
22                      cuenta := cuenta+1;
23                  end if;
24              end if;
25              div_clk <= cuenta(25);
26          end process;
27  end Behavioral;

```

Figura 7: Código VHDL del divisor de frecuencia que fija la velocidad de la máquina.

Finalmente, la entidad de nivel superior `Practica3` instancia los cinco módulos anteriores y los interconecta mediante señales internas, además de exponer el estado presente hacia los LEDs para

poder observarlo. Su código se muestra en la figura 8.

```

1 library IEEE;
2 use IEEE.STD_LOGIC_1164.ALL;
3
4 -- Practica 3: Construccion de Maquinas de Estados usando Memorias
5 -- Direccionamiento por Trayectoria
6 --
7 --           +-----+           +-----+           +-----+
8 --   Q1Q2Q3 ->| concatena |-- 6 -->| memoria |-- 7 -->| divisor |-- 4 -->
9 --   S1..S4
10 --   EdoPres->| dor_datos  | dir   | 64 x 7   | data  | _datos  |
11 --           ^   +-----+           +-----+           +-----+
12 --           |                                           | liga (3)
13 --           |           +-----+                                           |
14 --           +-----+| register2 |<-----+
15 --                   +-----+
16 --                   | CLK
17 --           CLK50 ----->| div_frec  |
18 --                   +-----+
19
20 entity Practica3 is
21     Port( CLK50 : in STD_LOGIC;                --Reloj de 50 MHz de la
22           tarjeta
23           RESET : in STD_LOGIC;                --Activo en bajo (KEY0)
24           Q1 : in STD_LOGIC;
25           Q2 : in STD_LOGIC;
26           Q3 : in STD_LOGIC;
27           S1 : out STD_LOGIC;                  --Logica estandar
28           S2 : out STD_LOGIC;                  --Logica estandar
29           S3 : out STD_LOGIC;                  --Logica negada (activa en
30           S4 : out STD_LOGIC;                  --Logica negada (activa en
31           ED0 : out STD_LOGIC_VECTOR(2 downto 0) --Estado presente
32           );
33 end Practica3;
34
35 architecture Estructural of Practica3 is
36     component div_frec is
37         Port( reloj : in std_logic;
38               div_clk : out std_logic);

```

```

39  end component;
40
41  component concatenador_datos is
42      Port( entradaA : in STD_LOGIC_VECTOR(2 downto 0);
43            entradaB : in STD_LOGIC_VECTOR(2 downto 0);
44            salida : out STD_LOGIC_VECTOR(5 downto 0));
45  end component;
46
47  component memory is
48      Port( dir : in STD_LOGIC_VECTOR(5 downto 0);
49            data : out STD_LOGIC_VECTOR(6 downto 0));
50  end component;
51
52  component divisor_datos is
53      Port( entrada : in STD_LOGIC_VECTOR(6 downto 0);
54            liga : out STD_LOGIC_VECTOR(2 downto 0);
55            salidas : out STD_LOGIC_VECTOR(3 downto 0));
56  end component;
57
58  component register2 is
59      Port( CLK : in STD_LOGIC;
60            RESET : in STD_LOGIC;
61            DATA_IN : in STD_LOGIC_VECTOR(2 downto 0);
62            DATA_OUT : out STD_LOGIC_VECTOR(2 downto 0));
63  end component;
64
65  signal clk_fsm : std_logic;
66  signal entradas : std_logic_vector(2 downto 0);  --Q1 Q2 Q3
67  signal edo_pres : std_logic_vector(2 downto 0);
68  signal direccion : std_logic_vector(5 downto 0);
69  signal palabra : std_logic_vector(6 downto 0);
70  signal liga : std_logic_vector(2 downto 0);
71  signal salidas : std_logic_vector(3 downto 0);  --S1 S2 S3 S4
72
73  begin
74
75      entradas <= Q1 & Q2 & Q3;
76
77      U1 : div_freq
78          port map( reloj => CLK50,
79                  div_clk => clk_fsm);
80
81      U2 : concatenador_datos
82          port map( entradaA => entradas,

```

```

83         entradaB => edo_pres,
84         salida => direccion);
85
86 U3 : memory
87     port map( dir => direccion,
88             data => palabra);
89
90 U4 : divisor_datos
91     port map( entrada => palabra,
92             liga => liga,
93             salidas => salidas);
94
95 U5 : register2
96     port map( CLK => clk_fsm,
97             RESET => RESET,
98             DATA_IN => liga,
99             DATA_OUT => edo_pres);
100
101 S1 <= salidas(3);
102 S2 <= salidas(2);
103 S3 <= salidas(1);
104 S4 <= salidas(0);
105 EDO <= edo_pres;
106
107 end Estructural;

```

Figura 8: Código VHDL de la entidad de nivel superior que interconecta los módulos.

Asignación de pines

Para la implementación en la tarjeta DE10-Lite (dispositivo MAX 10 10M50DAF484C7G) se realizó la asignación de pines de la tabla 3. Las tres entradas se asignaron a interruptores deslizables, el *reset* al botón KEY0 (que la tarjeta entrega en bajo al ser presionado, lo cual coincide con la polaridad del registro), las cuatro salidas a los LEDs LEDR0–LEDR3 y el estado presente a LEDR7–LEDR9.

Tabla 3: Asignación de pines en la tarjeta DE10-Lite.

Señal	Pin	Recurso	Función
CLK50	PIN_P11	MAX10_CLK1_50	Reloj de 50 MHz
RESET	PIN_B8	KEY0	<i>Reset</i> activo en bajo
Q1	PIN_C10	SW0	Entrada Q_1
Q2	PIN_C11	SW1	Entrada Q_2
Q3	PIN_D12	SW2	Entrada Q_3
S1	PIN_A8	LEDR0	Salida S_1 (activa en 1)
S2	PIN_A9	LEDR1	Salida S_2 (activa en 1)
S3	PIN_A10	LEDR2	Salida $\overline{S_3}$ (activa en 0)
S4	PIN_B10	LEDR3	Salida $\overline{S_4}$ (activa en 0)
EDO[2]	PIN_B11	LEDR9	Bit 2 del estado presente
EDO[1]	PIN_A11	LEDR8	Bit 1 del estado presente
EDO[0]	PIN_D14	LEDR7	Bit 0 del estado presente

Análisis de resultados

El proyecto se compiló en Quartus Prime 18.1.0 Lite Edition sin errores. La tabla 4 resume los recursos utilizados y el desempeño reportado por la herramienta.

Tabla 4: Resumen de compilación del proyecto en Quartus Prime 18.1.

Parámetro	Valor
Elementos lógicos	61 / 49 760 (< 1 %)
Registros dedicados	31 / 49 760 (< 1 %)
Pines de E/S	12 / 360 (3 %)
Bits de memoria embebida	0 / 1 677 312 (0 %)
F_{max} (modelo 1200 mV, 85 °C)	180.34 MHz
Errores de compilación	0

Dos resultados de la tabla 4 merecen comentario. El primero es el conteo de 31 registros, que coincide exactamente con lo esperado del diseño: 28 corresponden al contador del divisor de frecuencia y los 3 restantes al registro de estado. Esto confirma que el único elemento secuencial del lazo de la máquina de estados es dicho registro, tal como plantea el esquema de direccionamiento por trayectoria. El segundo es que la herramienta reporta 0 bits de memoria embebida: la ROM de 448 bits no se mapeó a los bloques M9K del dispositivo, sino que se sintetizó como lógica combinacional dentro de los 61 elementos lógicos. Esto es razonable dado su tamaño reducido y no altera el comportamiento del circuito, aunque implica que el contenido de memoria quedó fijo en la configuración del FPGA y no puede reescribirse en tiempo de ejecución.

En cuanto a la frecuencia máxima, los 180.34 MHz reportados superan ampliamente los 50 MHz del oscilador de la tarjeta, por lo que el diseño opera con holgura. El analizador temporal emite una advertencia de incumplimiento de restricciones únicamente porque el proyecto no incluye un archivo .sdc; en ausencia de restricciones declaradas, Quartus asume un requisito por omisión de 1 GHz.

El funcionamiento del circuito se verificó directamente sobre la tarjeta DE10-Lite ante la profesora durante la sesión de laboratorio. Con todos los interruptores en cero, la máquina permanece indefinidamente en EST0 con LEDR0 encendido, comportamiento que coincide con las direcciones 0 a 3 de la tabla 2, cuya liga apunta al propio EST0. Al fijar $Q_1 = 1$ y $Q_2 = 1$ se observó el ciclo EST0 $\rightarrow$ EST1 $\rightarrow$ EST2 $\rightarrow$ EST0, avanzando un estado por segundo, congruente con las ligas de las direcciones 4, 10 y 16. Con $Q_1 = 1$ y $Q_2 = 0$ se recorrió la rama opuesta, EST0 $\rightarrow$ EST1 $\rightarrow$ EST3 $\rightarrow$ EST4, quedando la máquina en EST4 por el autolazo que impone $Q_3 = 0$, tal como indican las direcciones 32, 34, 36 y 38.

La observación más relevante se obtuvo con la máquina detenida en EST4. Al conmutar Q_3 de 0 a 1, los LEDs de salida cambiaron de inmediato (LEDR3 se encendió y LEDR0 también), mientras que los LEDs de estado conservaron el valor 100 hasta el siguiente flanco del reloj dividido, momento en el cual la máquina transitó a EST0. Este desfase entre el cambio de las salidas y el cambio del estado es precisamente el comportamiento esperado del esquema: las salidas provienen de forma combinacional de la palabra leída, y como la dirección incluye las entradas, modificar Q_3 cambia la localidad consultada de manera instantánea. El estado, en cambio, solo se actualiza cuando el registro captura la liga en el flanco de reloj. Se verificó también que al regresar Q_3 a 0 antes del flanco, la máquina permanece en EST4, lo que confirma que la decisión se resuelve con el valor de la entrada presente en el instante del flanco.

Respecto a las salidas con lógica negada, se comprobó que LEDR2 y LEDR3 permanecen encendidos durante la mayor parte de la operación y se apagan únicamente cuando $\overline{S_3}$ o $\overline{S_4}$ se activan, lo cual es consistente con el criterio de codificación adoptado y con los valores de la columna $S_1S_2S_3S_4$ de la tabla 2.

No se detectó ninguna discrepancia entre el comportamiento observado en la tarjeta y el previsto por la carta ASM de la figura 2.

Opcional: agregar la captura del RTL Viewer de Quartus para ilustrar la interconexión de los cinco módulos, y la captura del Pin Planner como respaldo de la tabla 3.

Conclusiones individuales

Omar Alfonso Zendejas Labias

Se cumplió el objetivo de la práctica. La máquina de cinco estados quedó implementada con una memoria de 64 localidades de 7 bits y funcionó en la tarjeta tal como lo predice la carta ASM, recorriendo las dos ramas de la decisión de EST1 y respetando los autolazos de EST3 y EST4. Lo que más me ayudó a entender el método fue ver en la tarjeta que al mover Q_3 estando en EST4 las salidas cambiaban de inmediato pero el estado no: ahí quedó claro que la dirección de la memoria se arma con el estado y las entradas juntos, y que el registro es lo único que espera al flanco de reloj. También me sirvió comparar este método con el de la práctica anterior, donde había que obtener las ecuaciones por mapas de Karnaugh; aquí cambiar el comportamiento de la máquina sería solo reescribir renglones de la tabla. Queda pendiente definir con el equipo si el tamaño que se reporta son los 448 bits que da la fórmula del manual o los 280 bits de las localidades que realmente se usan.

Emiliano García Olvera

Conclusión individual de Emiliano. Siguiendo el formato: un solo párrafo, en primera persona, indicando si se cumplió el objetivo y relacionándolo con los resultados concretos del reporte (tabla 2, tabla 4 o lo observado en la tarjeta), sin narrar la experiencia personal.

Bibliografía

- [1] J. Savage Carmona, G. Vázquez y N. E. Chávez Rodríguez, *Diseño de microprocesadores*. Ciudad de México, México: Facultad de Ingeniería, Universidad Nacional Autónoma de México, 2017. [En línea]. Disponible: <http://profesores.fi-b.unam.mx/luist/archivos/MicroprocesadoresSavage.pdf>
- [2] M. M. Mano y M. D. Ciletti, *Diseño digital*, 5a ed. Naucalpan de Juárez, México: Pearson Educación, 2013.